

Vibe Studio

바이브 코더의 "조작"까지 설계한다

클릭으로 용어를 대신하는 **질문 모드**에 더해, ① 멀티에이전트를 한눈에 보는 **오케스트레이션 보드**, ② 직관적인 **간결 프롬프트 입력**, ③ 내장 로컬 LLM의 **함수 단위 검수**로 **유료 토큰을 아끼는** 파이프라인을 더한 통합 기획.

멀티에이전트 가시성

순서 완료 진행 상태

간결 직관 입력

로컬 LLM 함수 검수

토큰 절약 다이제스트

보안·누락·난립 예방

01 이번 업데이트 — 조작 경험 3대 축

"무엇을 가리킬지(질문 모드)"에 이어, 바이브 코더가 실제로 다루는 화면·입력·검수를 쉽게 만든다.

■ 한눈에 보는 에이전트

여러 에이전트가 동시에 돌 때 완료/진행중/대기 (순서)를 색과 배치로 즉시 구분. 누가 무엇을, 어떤 순서로 하는지 한 화면에.

👉 간결한 프롬프트

복잡한 패널 없이 한 줄 입력바 + 칩. 클릭한 요소가 칩으로 들어오고 이어서 말만 쓰면 전송. 초보자도 망설임 없이.

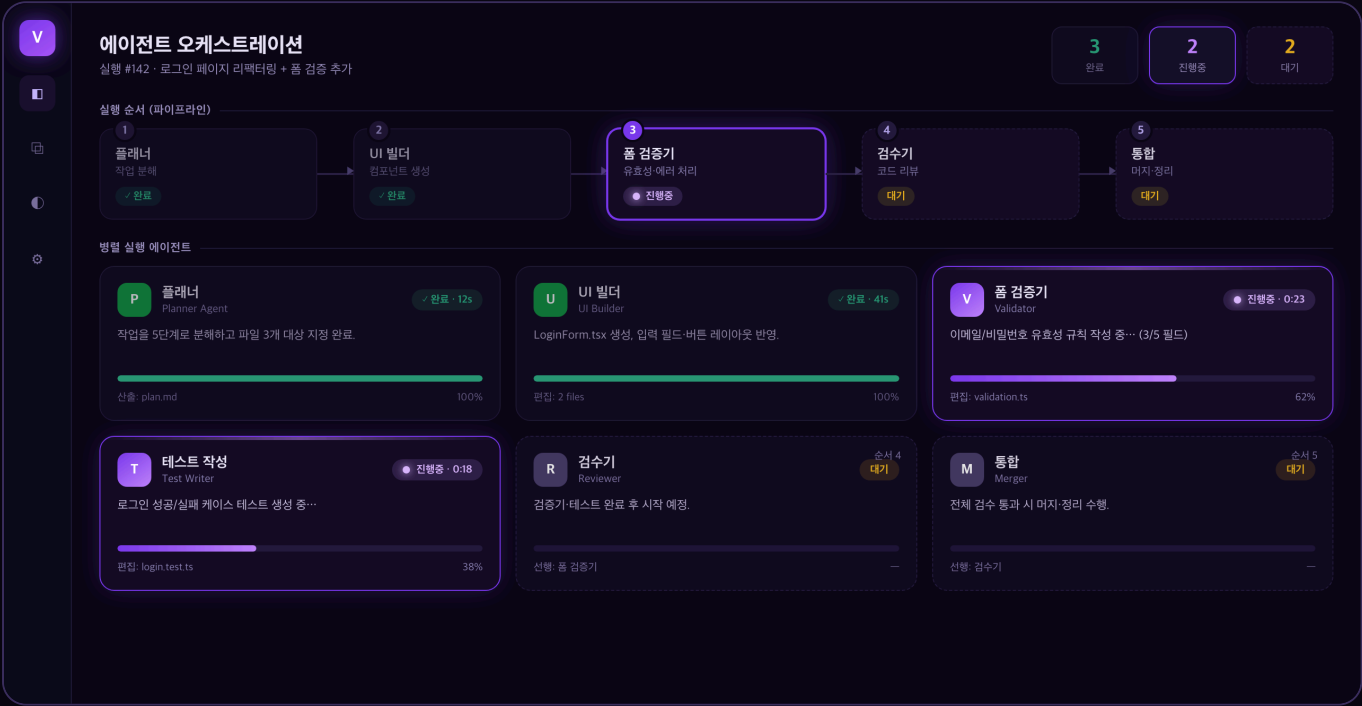
🕒 토큰 아끼는 검수

내장 로컬 LLM이 함수 하나씩 읽어 설명·검증 → 요약만 클라우드 에이전트로. 유료 토큰↓, 보안·누락·코드 난립↓.

일관 원칙: 지식이 없어도, 화면이 복잡하지 않아도, 비용이 커지지 않아도 — 바이브 코더가 끝까지 스스로 진행할 수 있게 한다.

02 멀티에이전트 오케스트레이션 보드

동시에 도는 에이전트의 상태·순서·의존관계를 한 화면에서 파악.



목업 ① — 상단 요약(완료 3·진행중 2·대기 2), 파이프라인 순서 레인, 병렬 실행 카드

상태를 색으로

- 완료 초록·흐림 · 진행중 보라 글로우 · 대기 노랑·점선. 스캔하듯 즉시 구분.

순서를 레인으로

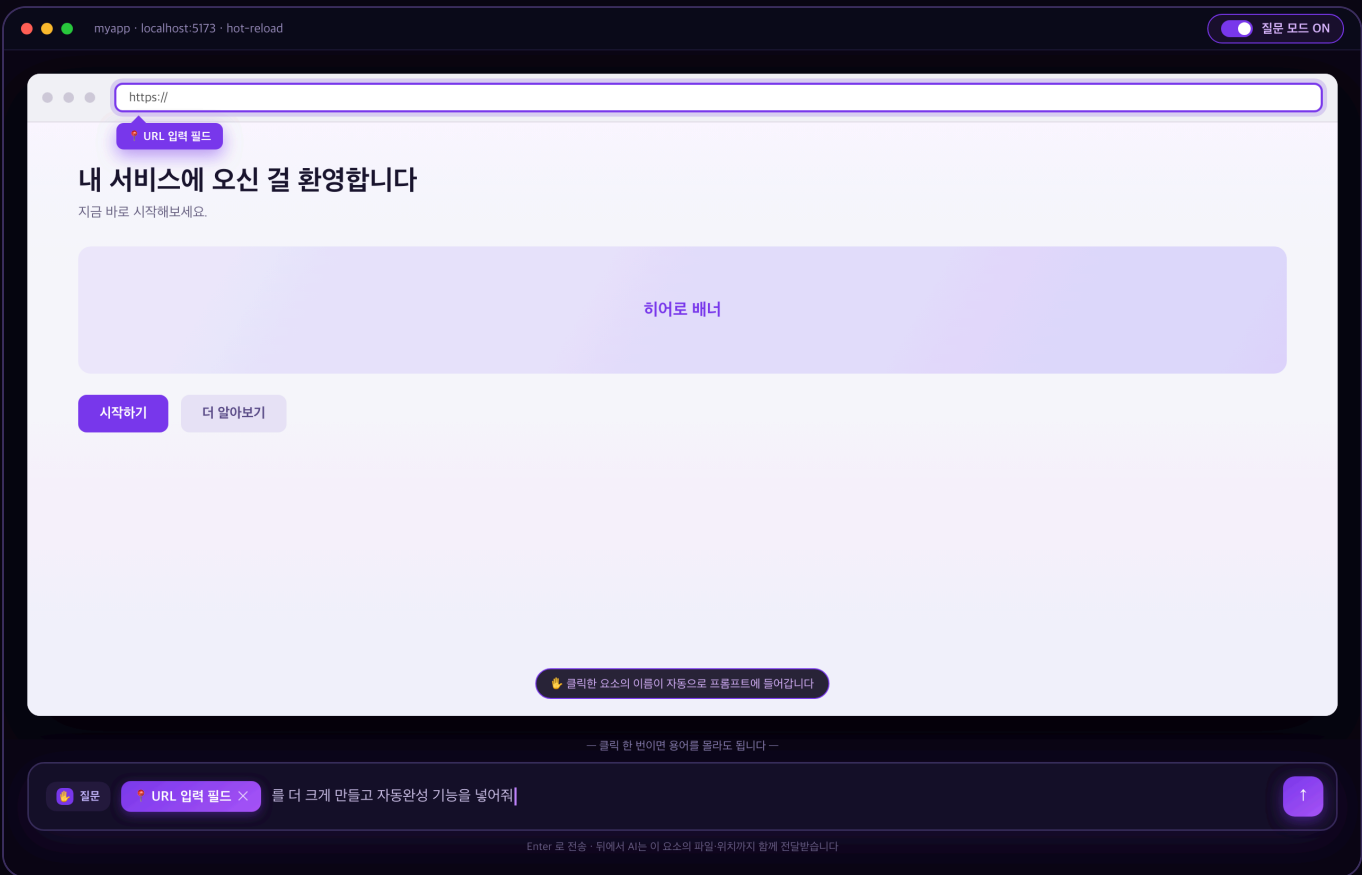
번호 뱃지 + 화살표 연결로 실행 순서(1→5)를 시각화. 지금 어디까지 왔는지 명확.

의존을 명시

대기 카드에 "선행: 폼 검증기"처럼 무엇을 기다리는지 표기 → 멈춘 이유가 보임.

03 간결한 프롬프트 입력 + 질문 모드

입력은 한 줄이 원칙. 클릭한 요소가 칩으로 들어오고, 사용자는 의도만 이어 쓴다.



목업 ② — hot-reload 프리뷰에서 URL 창 클릭 → "URL 입력 필드" 칩이 프롬프트에 삽입, 이어서 자연어 작성

👉 클릭 = 용어 입력

- 질문 모드 ON → 요소 호버 시 **이름 미리보기**, 클릭 시 칩 삽입
- 사용자 화면엔 "URL 입력 필드"만, 뒤에서 시엔 **파일 위치**까지 전달

👉 군더더기 없는 입력바

- 패널·설정 없이 **칩 + 텍스트 + 전송** 한 줄
- 칩은 **삭제·수정** 가능, 여러 개 삽입 가능(다중 선택)
- Enter 전송 — 초보자의 인지 부담 최소화

왜 간결해야 하나: 바이트 코더는 옵션이 많을수록 막힌다. "가리키고 → 말하고 → 엔터" 3동작으로 압축해 **진입장벽을 0에 가깝게**.

04 내장 로컬 LLM — 함수 단위 검수 & 토큰 절약

코드 전체를 클라우드 모델에 넘기지 않는다. 로컬이 함수별로 읽고 요약(digest)만 올려 토큰을 아낀다.

내장 로컬 LLM · 함수 단위 코드 검수

코드 전체가 아니라 함수 하나씩 읽어 설명·검증 → 종합 → 클라우드 에이전트로 요약만 전달

온디바이스 로컬 LLM

토큰 소비 없음

function validateUrl(input)

입력 문자열이 유효한 URL인지 정규식으로 검사한다. 프로토콜 누락 시 https 를 자동 보정.

구현 정상

빈 입력 예외 누락

function handleSubmit(e)

폼 전송 시 URL 검증 후 navigate 호출. 사용자 입력을 그대로 라우팅에 사용.

구현 정상

미검증 입력 사용(보안)

function formatDisplay(url)

표시용으로 프로토콜 www 를 제거해 축약. 순수 함수, 부작용 없음.

구현 정상

오류 없음

종합 → 클라우드

종합 모델

함수별 설명·검증 결과를 파일 단위 요약으로 통합

↓

파일 요약(digest)

handleSubmit: 미검증 입력 라우팅 → 보안 위험

validateUrl: 빈 입력 예외 누락

중복/불필요 유틸 없음 · 구조 양호

여기까지 로컬(무료) · 아래부터 클라우드

Claude / GPT 에이전트

보안 관련 텍스트 작업 중

토큰 ↓ 80%

전체 코드가 아닌 요약만 받아 수정 지시. 사용자의 유료 토큰을 아끼면서 보안·누락·코드 난립을 예방.

보안 예방

미검증 입력·취약 패턴을 요약에 표시해 에이전트가 먼저 처리

누락·유틸 점검

예외 처리·필요 유틸 빠짐을 함수 단위로 포착

코드 난립 제한

중복·불필요 파일 증식(우후죽순)을 사전 감지·억제

목업 ③ — 함수별 설명+검증(구현?/오류?) → 종합 모델 → 파일 요약 → 클라우드 에이전트(토큰 ↓80%)

① 함수 분해

파일을 함수 단위로 분리

② 로컬 설명·검증

함수마다 설명 + 구현/오류 판정

③ 종합 모델

파일 단위 요약으로 통합

④ 클라우드 전달

요약만 에이전트로 → 수정

왜 "함수 단위"인가

- 전체 파일을 한 번에 읽으면 맥락이 흐려지고 토큰·비용 폭증
- 함수는 작고 독립적 → 작은 로컬 모델도 정확히 설명·검증 가능
- "제대로 구현됐나 / 오류는 없나"를 지점별로 국소 판정

무엇을 예방하나

- 보안 — 미검증 입력·취약 패턴을 요약에 표시해 먼저 처리
- 누락·유틸 — 예외 처리·필요 유틸 빠짐을 함수 단위로 포착
- 코드 난립 — 중복·불필요 파일 증식(우후죽순) 사전 감지·억제

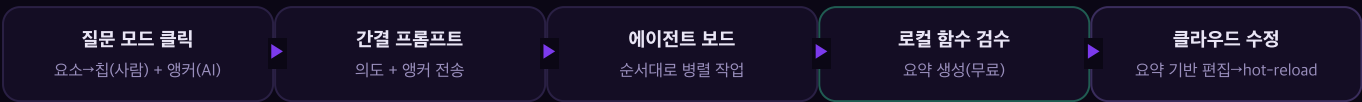
핵심 이득: 무거운 읽기는 로컬(무료)이, 판단·수정은 클라우드(유료)가 — 역할을 나눠 사용자의 Claude·GPT 토큰을 크게 아끼면서 품질은 지킨다.

Vibe Studio · Spec v3

03

05 통합 데이터 흐름

세 축이 하나의 루프로 연결된다.



06 로드맵(업데이트)

- **Phase 0 — 질문 모드 + 간결 입력 · 2~3주**
 - 프리뷰 오버레이·접근성 트리 라벨·칩 삽입(앞 기획 유지)
 - 한 줄 입력바 UX 확정
- **Phase 1 — 에이전트 오케스트레이션 보드 · 3~4주**
 - 상태(완료/진행중/대기)·순서 레인·의존관계 시각화
 - 실시간 진행률·산출물·경과 시간 스트리밍
- **Phase 2 — 로컬 LLM 함수 검수 파이프라인 · 4~6주**
 - 온디바이스 소형 모델 서빙 + 함수 분해기(AST)
 - 함수별 설명·구현/오류 판정 → 종합 모델 → digest
 - 클라우드 에이전트 연동(요약 주입)·토큰 절감 측정
- **Phase 3 — 프레임워크·플랫폼 확장 · 이후**
 - Vue/Svelte/순수 HTML, 앱 대상 확대

07 착수 전 결정

① 로컬 모델
어떤 온디바이스 모델? 크기 vs 정확도. ▶ 소형 (3~8B)부터, 함수 단위라 충분

② 보드 실시간성
폴링 vs 스트리밍. ▶ 이벤트 스트리밍 권장

③ MVP 타킷
▶ React(Vite) 우선 유지

다음 단계 제안: Phase 0~1(질문 모드 + 간결 입력 + 에이전트 보드)로 보이는 데모를 먼저 완성하고, Phase 2에서 로컬 검수로 "토큰 N% 절감"을 수치로 증명.